

Week 1: PyTorch Foundations

Theme:	From sklearn to PyTorch — same ideas, more flexibility
Time Commitment:	8–12 hours
Suggested Split:	3h reading/video, 6h coding, 1h review

Learning Objectives

By the end of this week, you should be able to:

- Manipulate PyTorch tensors and understand the autograd system
- Write a complete training loop from scratch (no `model.fit()`)
- Train a simple feedforward network on MNIST achieving $\geq 97\%$ accuracy
- Move computations to GPU and understand device management

Primary Resources (Must-Do)

[BLOG/DOCS] PyTorch Official 60-Minute Blitz

Type: Tutorial | **Time:** 60–90 min | **Difficulty:** Beginner

URL: https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html

Why this resource: The canonical starting point. Covers tensors, autograd, neural network basics, and a full classification example in a tight, well-paced format.

Focus on: All four sections in order. Don't skip the autograd section — it's the foundation for everything that follows.

[VIDEO] Karpathy: “The spelled-out intro to neural networks and backpropagation”

Type: YouTube Lecture | **Time:** 2.5 hours (watch first 1 hour) | **Difficulty:** Beginner-Intermediate

URL: <https://www.youtube.com/watch?v=VMj-3S1tku0>

Why this resource: Andrej Karpathy builds backprop from absolute scratch. After this video, autograd will feel obvious instead of magical.

Focus on: The first hour through the implementation of `Value` class. The micrograd library he builds is essentially a tiny PyTorch.

[BLOG/DOCS] PyTorch Documentation: `nn.Module`, Optimizers, `DataLoader`

Type: Documentation | **Time:** 30–45 min (skim) | **Difficulty:** Beginner

URLs:

- <https://pytorch.org/docs/stable/generated/torch.nn.Module.html>
- <https://pytorch.org/docs/stable/optim.html>
- <https://pytorch.org/docs/stable/data.html>

Why this resource: Bookmark these. You'll return to them constantly throughout the program. Read enough to know what's available; come back as needed.

[VIDEO] NPTEL: Deep Learning by Prof. Mitesh Khapra (IIT Madras)

Type: Video Lecture Series | **Time:** Pick 2–3 hours of relevant lectures | **Difficulty:** Beginner

URL: <https://nptel.ac.in/courses/106106184/>

Why this resource: Excellent foundational coverage in clear English with strong mathematical grounding.

Focus on: Lectures on feedforward networks, backpropagation, and gradient descent variants.

This Week's Deliverable

Build a feedforward neural network that classifies MNIST digits with $\geq 97\%$ test accuracy. Your repository must include:

- A custom `Dataset` class (don't just use `torchvision.datasets.MNIST` directly — subclass it or wrap it)
 - A manual training loop with logging of train/val loss and accuracy per epoch
 - Train/val/test split (use a portion of training data for validation)
 - Saving and loading model checkpoints (`torch.save`, `torch.load`)
 - A `README.md` explaining your design choices and final results
- Push to your GitHub repository before next Saturday's group call.

Key Concepts Checklist

You should be able to confidently answer all of the following:

- What does `loss.backward()` actually do? Where do gradients live?
- Why do we need `optimizer.zero_grad()` every iteration?
- What's the difference between `model.train()` and `model.eval()`?
- When and why do we use `torch.no_grad()`?
- How does `DataLoader` differ from a Python list of batches?
- What's the role of `nn.Module`? Why do we subclass it?

Stretch Resources (Optional)

For mentees who finish early or want deeper understanding:

- **Daniel Bourke's PyTorch course** (free, YouTube): https://www.youtube.com/watch?v=Z_ikDlimN6A
- **Sebastian Raschka's "Machine Learning with PyTorch and Scikit-Learn"** — Chapters 12–13 if accessible
- **Dive into Deep Learning** (free book, PyTorch sections): <https://d2l.ai/>

Debugging & Reference

- **PyTorch Forums:** <https://discuss.pytorch.org/> — search before posting
- **Common errors guide:** <https://pytorch.org/docs/stable/notes/faq.html>
- **Stack Overflow tag:** `[pytorch]`